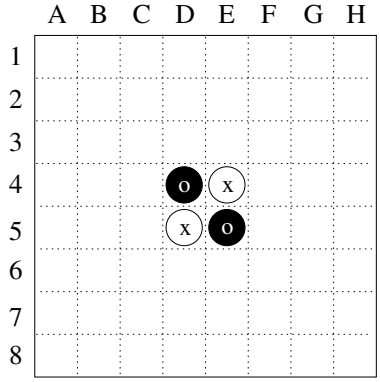
Practical ex. 2: Minimax y $\alpha - \beta$ algorithms. Heuristics for games.

1. Explanation of the problem

The first objective of this exercise, to be developed in two sessions, is to understand and implement the $\alpha - \beta$ algorithm for resolution of game trees. The algorithm should be implemented as generally as possible, but it will be tested applying it to the game of Otello. In addition you must devise several game heuristics and make them compete against each other, either between those made by the components of the same group or against those made by other groups.

The rules of Otello are: we have a square board made by 64 squares (8 rows and 8 columns). 64 tokens exist with the shape of a coin and with different colors or symbols on each side (as in real coins). In our case, let's say one face white and the other black. The initial configuration is as shown in the figure.



Each player is associated with a color (face) of the tokens. Starting with the player who plays with black, each player places a token in any of the free squares with a restriction: when doing so, at least one token of the opponent's color should be reversed. A token must be reverted if it is aligned horizontally, vertically, or diagonally between two of the opposite color, and also the whole line between them is continuously occupied (without holes) by tokens of the same color. This means e.g. that from the starting position, the first move of black could only be either to E3 (then, flipping the white token at E4), to C5 (flipping D5), to F4 (flipping E4) or to D6 (flipping D5).

If a player cannot place any token of his color according to these rules, he must pass, and only in that case is he allowed to do so. The game ends when all the 64 tokens are on the board, or when none of the two players can put more tokens. The winner is the player whose color is displayed at that time in more tokens on the board, or the two players tie in case of equality in number of tokens.

The result you must submit is:

- The implementation of the $\alpha - \beta$ algorithm and its test in the game.
- The implementation of at least two heuristics and their test in the game playing one against the other.
- A file `explanation.txt` in ASCII text with the compilation instructions and results obtained with each heuristic.

The source code SUFFICIENTLY COMMENTED will have to be submitted, too.

Appendix: C++ API

Some data type are defined and a class `Board` is provided to help with the implementation.
The data types are:

- Type Square: `typedef pair<char,char>Square`. It is a pair of two characters that represents a square of the board. To be valid, the first character must be '1',...,'8' (the row), and the second, 'A',...,'H' (the column). Notice that digits are represented as characters, not as integers.
- Type Sqlist: `typedef list<Square>Sqlist`. A list of squares.
- Type Token: defined as an enumerated type `enum Token {x,o,e }`. Letter o represents a black square, x represents a white one and the e an empty square.

- Type Player: defined as an enumerated type `enum Player {Bob,Alice}`. By convention, Alice plays with black tokens (token 'o') and therefore opens the game.
- Type Move: `typedef pair<Player,Square>Move`. It represents a move where the first component is the player who is doing the move and the second the square where he will put the token of his color.

Auxiliary functions are:

- `string Name(Player pl)`: Takes a player (as a Player type) and returns a string with his/her name. (This is meant to be used just for printing/debugging).
- `Player Opponent(Player pl)`: Takes a player (as a Player type) and returns his/her opponent, also as Player type.
- `Token Tk(Player pl)`: Takes a player (as a Player type) and returns the value (with Token type) of the token he/she plays with.
- `char Symbol(Token tk)`: Takes a token (as a Token type) and returns a character. (This is meant to be used just for printing/debugging).
- `bool ValidSquare(Square sq)`: Takes a square and returns true if it is valid and false otherwise. See below the restrictions to the characters that can be used in the Square type.
- `bool ValidPlayer(Player pl)`: Takes a player (as a Player type) and returns true if it is one of the two valid players and false otherwise.

The provided class is called `Board` and its public interface is:

- `Board()`: Creates a board in its initial state (i.e. two 'o' tokens in ('4','D') and ('5','E') and two 'x' tokens in ('5','D') and (4,'E')).
- `void Show(ostream &outst)`: Shows in the console (passing cout or cerr) or in any other output stream the current state of the board.
- `Token Content(Square sq)`: Returns the token (x,o,e) which is currently at the square passed as argument.
- `bool IsLegal(Move mv)`: Returns `true` if the move passed as argument can be executed in the current state of the board and `false` if not.
- `void MakeMove(Move mv)`: Makes the move passed, if it is allowed (There is no need to check for that in advance, this function makes it) which changes the internal state of the board (the object that called this function).
- `Board TryMove(Move mv,bool &legal)`: If the move passed as first argument is allowed (this is internally checked) make a copy of the current board in a new one, executes the move in that copy and returns it. Therefore, the object that calls this function remain unchanged. The second argument`legal`, passed by reference, returns with the obvious value depending on the move can be executed or not.
- `bool GameOver(Player &winner,unsigned &winnerpoints,unsigned &looserpoints)`: Returns `true` if the game is over, i.e. if none of the players can make any movement (either because of blocking or because the board is full) and `false` otherwise. Moreover, if the game is over the variables passed by reference return with respectively the winner (as Player type), the numbers of tokens the winner has and such number for the looser (both with type `unsigned`).
- `Sqlist ValidMoves(Player pl)`: Returns a list of squares in which the player passed as argument is allowed to place a token (of course with his/her assigned color).

The heuristic function is defined in files `heuristic.h/cpp`. The non-informative heuristic is provided (always returns 0) and the exercises requires the alumni write other or others better heuristics using those functions of the `Board` class considered useful (mainly, `Content` to search which tokens are in the board and where they are placed). At least two heuristics should be programmed to make one play against the other. To do so the heuristic function can be implemented as

```
#include "heuristic.h"

float h(Player pl,Board b)
{
 if (pl==Bob)
 {
  // Program one heuristic here...
  // return ...
 }
 else
 {
  // Program a different one here...
  // return ...
 }
}
```

Also, the main programs plays with a minimax algorithm, defined in `Minimax.h/cpp`. The implementation is fully commented. You must copy these two files in others, `Alfabeta.h/cpp` and make the necessary modifications to write an $\alpha - \beta$ algorithm, testing it later in the main program as a substitute of minimax.